Faculty of Engineering and The Built Environment

University of Witwatersrand,

Johannesburg, South Africa.

School of Mining Engineering MINN2020A:

Computer Programming for Mining

PROJECT

18th August 2026

DESIGN, DEVELOPMENT AND DOCUMENTATION OF AN AFRICAN CRITICAL MINERALS APPLICATION

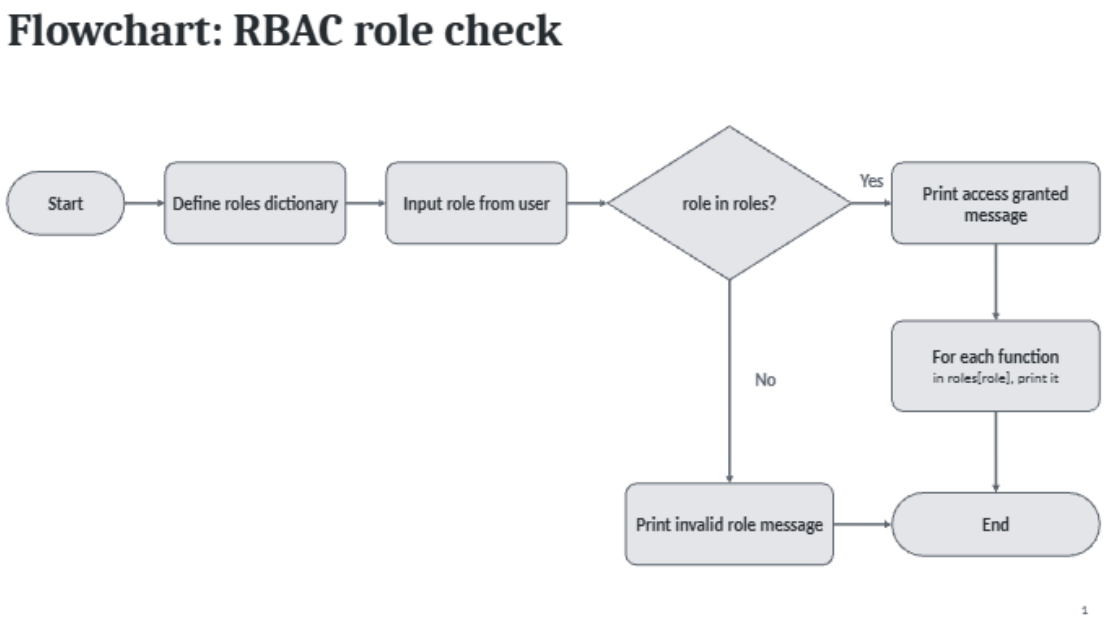
Dr. Milka Madahana and Mr. Tebatso Phala

GROUP 17

Name and surname	Student number	% Contribution	Signature
USENATHI KETSHINI	2839943	100	U.A Kitshini
LELAKA NKWADI	3024254	100	N.K.G LELAKA
LESHABA ANTONET	2845754	100	A.M Leshaba
SAMKELO GUMBI	3002328	100	S.S GUMBI

a)

Flowchart



Psedocode

START

SET roles TO a dictionary where each key is a role name and

each value is a list of functions that role can access:

"Administrator" -> [8 functions, including "Manage users"]

"Safety Officer" -> [5 functions]

"Mining Engineer" -> [6 functions]

"Maintenance Engineer" -> [4 functions]

"Manager" -> [7 functions]

INPUT role FROM user ("Enter your role: ")

IF role EXISTS AS A KEY IN roles THEN

DISPLAY "Access granted for {role}. You can:"

FOR EACH function IN roles[role] DO

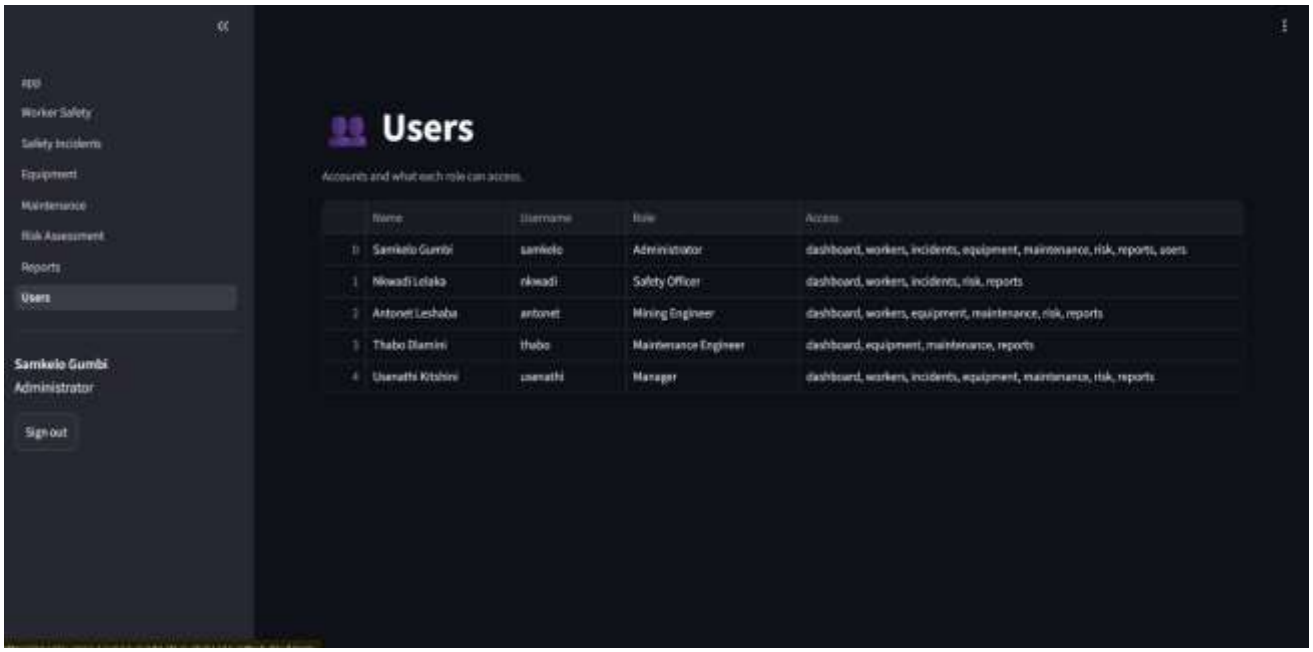
```
    DISPLAY " - " + function
  END FOR
ELSE
  DISPLAY "Invalid role. Please try again."
END IF
END
```

Code

```
# Role-Based Access Control (RBAC) demo
roles = {
  "Administrator": [
    "View dashboard", "Worker safety data", "Safety incidents",
    "Equipment data", "Maintenance data", "Risk assessment",
    "Generate reports", "Manage users" ],
  "Safety Officer": [
    "View dashboard", "Worker safety data", "Safety incidents",
    "Risk assessment", "Generate reports"],
  "Mining Engineer": [
    "View dashboard", "Worker safety data", "Equipment data",
    "Maintenance data", "Risk assessment", "Generate reports" ],
  "Maintenance Engineer": [ "View dashboard", "Equipment data", "Maintenance data",
    "Generate reports" ],
  "Manager": [
    "View dashboard", "Worker safety data", "Safety incidents",
    "Equipment data", "Maintenance data", "Risk assessment",
    "Generate reports" ]
role = input("Enter your role: ")
if role in roles:
```

```
    print("Access granted for {role}. You can:")
    for function in roles[role]:
        print(" -", function)
else:
    print("Invalid role. Please try again.")
```

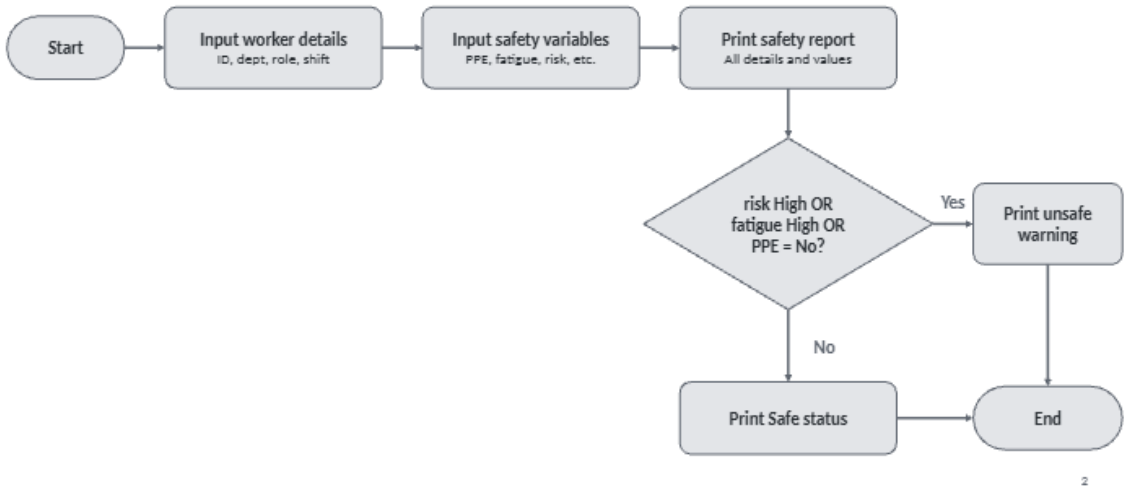
Graphical User Interface



b)

Flowchart

Flowchart: worker safety report



2

PseudoCode

START

// Collect worker details

INPUT worker_id

INPUT department

INPUT job_role

INPUT shift

// Collect safety variables

INPUT ppe (Yes/No)

INPUT training (Complete/Incomplete)

INPUT fatigue (Low/Medium/High)

INPUT observations

INPUT near_misses

INPUT incidents

INPUT risk (Low/Medium/High)

```
// Print the report
DISPLAY "--- Worker Safety Report ---"
DISPLAY worker_id, department, job_role, shift
DISPLAY ppe, training, fatigue, observations,
      near_misses, incidents, risk

// Safety check
IF lowercase(risk) = "high"
  OR lowercase(fatigue) = "high"
  OR lowercase(ppe) = "no" THEN
   DISPLAY "Warning: Unsafe condition detected!"
ELSE
   DISPLAY "Worker status: Safe"
END IF
END
```

Code

```
#Code for b
# Collect worker details
worker_id = input("Enter Worker ID: ")
department = input("Enter Department: ")
job_role = input("Enter Job Role: ")
shift = input("Enter Shift: ")

# Collect safety variables
ppe = input("PPE compliance (Yes/No): ")
training = input("Safety training status (Complete/Incomplete): ")
```

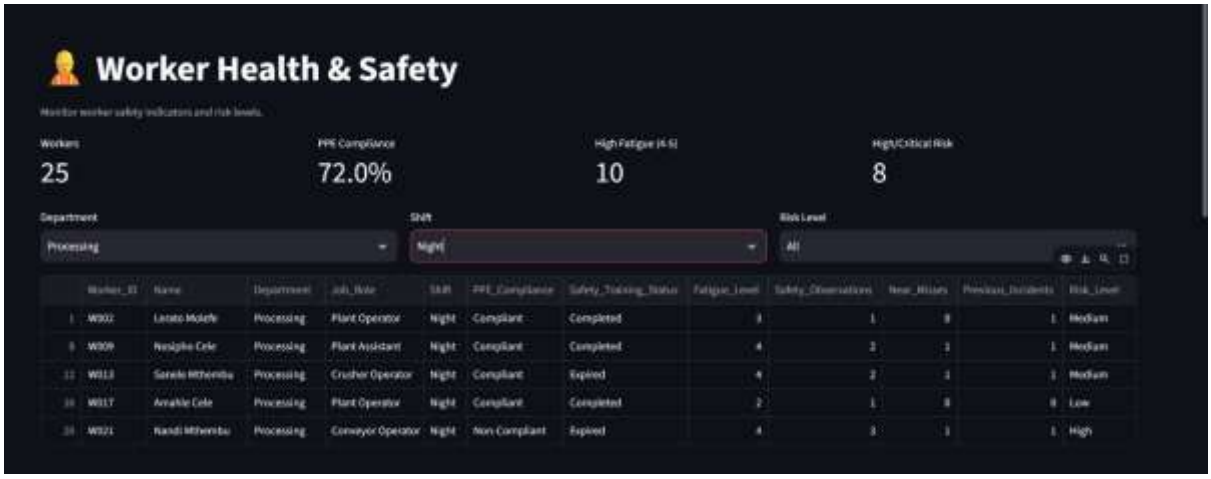
```
fatigue = input("Fatigue level (Low/Medium/High): ")
observations = input("Safety observations: ")
near_misses = input("Number of near misses: ")
incidents = input("Previous incidents: ")
risk = input("Risk level (Low/Medium/High): ")

# Check conditions
print("\n--- Worker Safety Report ---")
print("Worker ID:", worker_id)
print("Department:", department)
print("Job Role:", job_role)
print("Shift:", shift)

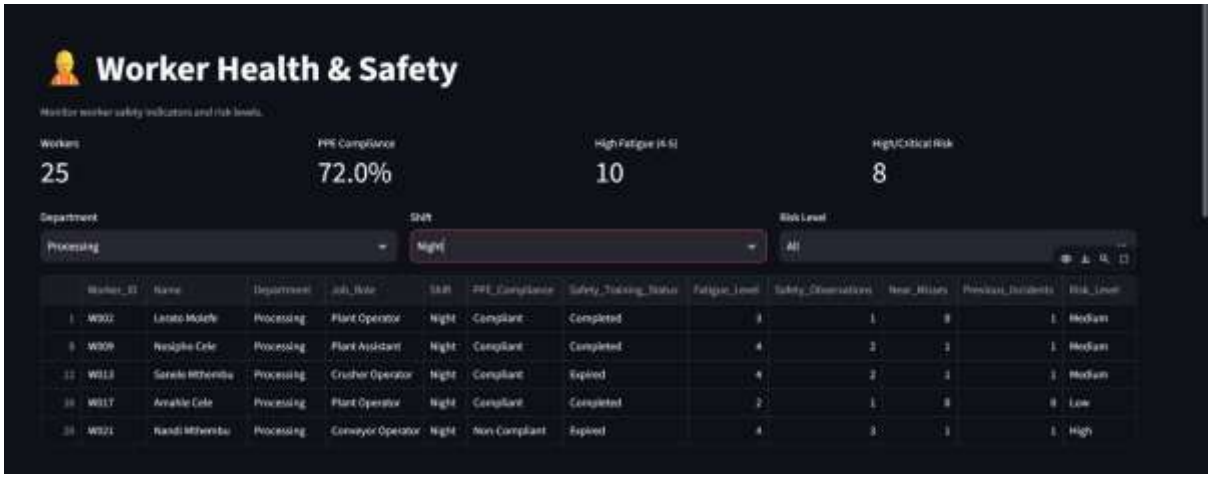
print("PPE:", ppe)
print("Training:", training)
print("Fatigue:", fatigue)
print("Observations:", observations)
print("Near Misses:", near_misses)
print("Incidents:", incidents)
print("Risk Level:", risk)

if risk.lower() == "high" or fatigue.lower() == "high" or ppe.lower() == "no":
    print("\n ⚠️ Warning: Unsafe condition detected!")
else:
    print("\n ✅ Worker status: Safe")
```

Graphical User Interface



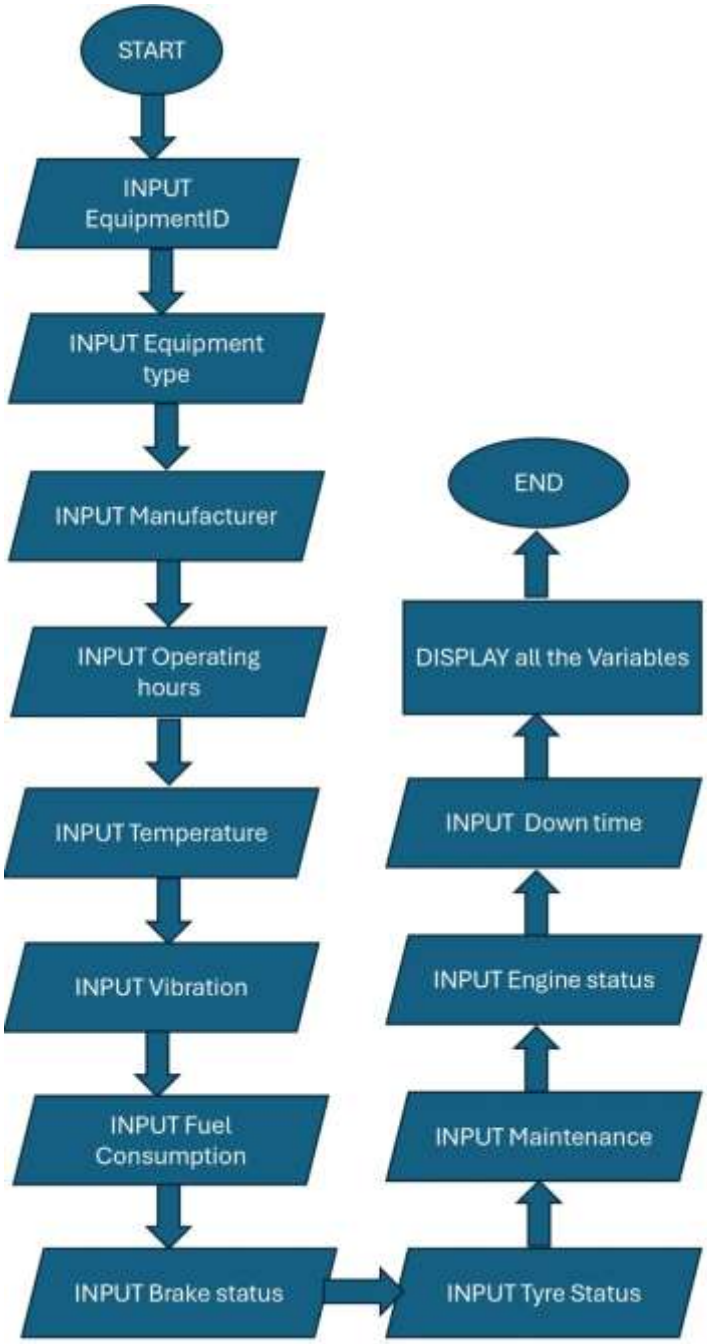




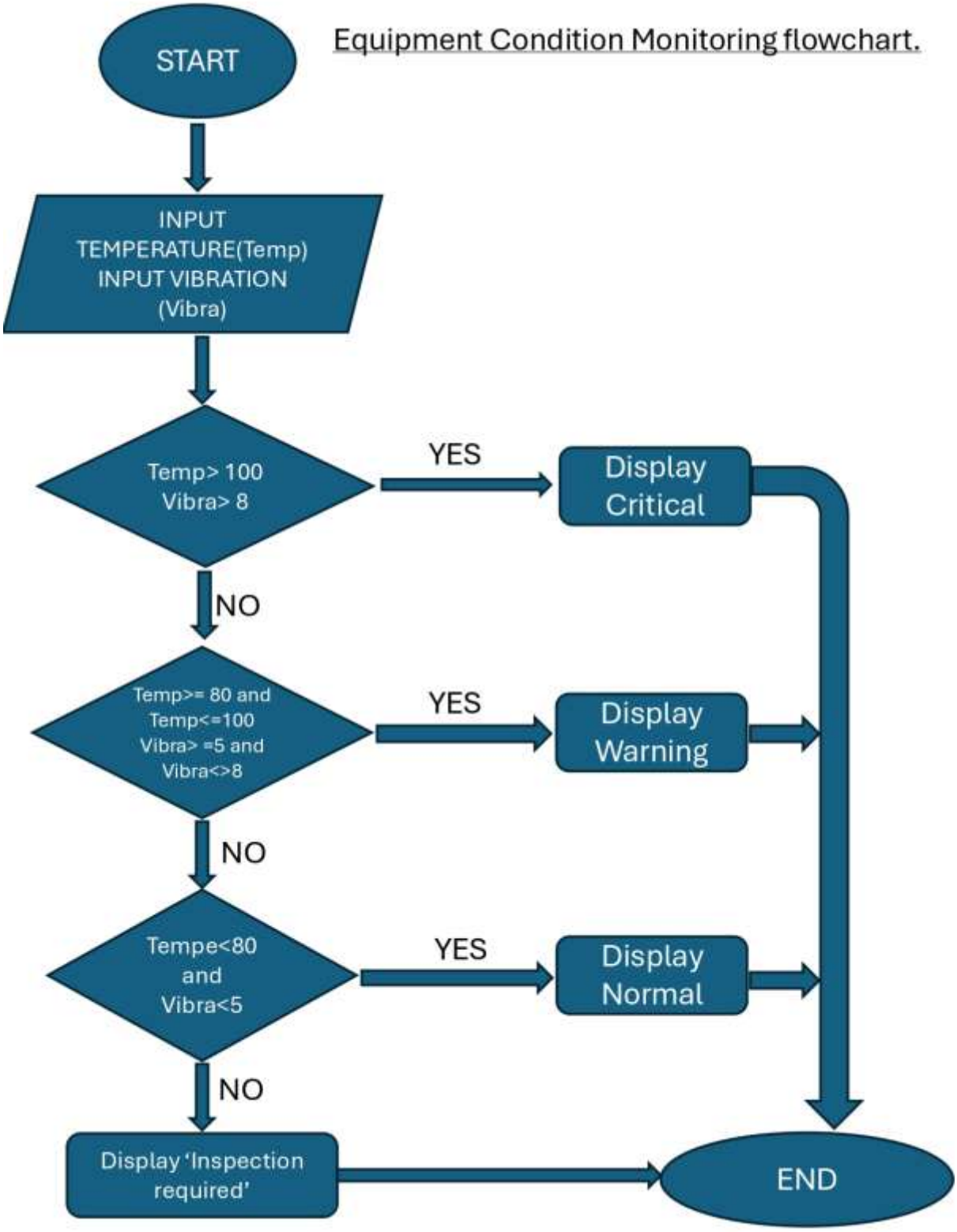

Fatigue Level Distribution (1-5)

1. **FLOWCHART**

d. Equipment Database



2.


Equipment Condition Monitoring flowchart.
START
INPUT TEMPERATURE(Temp)
INPUT VIBRATION (Vibra)
Temp> 100
Vibra> 8
YES
Display Critical
NO
Temp>= 80 and Temp<=100
Vibra> =5 and Vibra<>8
YES
Display Warning
NO
Tempe<80 and Vibra<5
YES
Display Normal
NO
Display 'Inspection required'
END

3. **Pseudocode**

#Equipment Database
Start
Declare variables for all equipment details
Input all the variables for the equipment
Read the values
Display the entered information
End

EQUIPMENT CONDITION MONITORING
Temperature = int(input("Enter Temperature: "))
Vibration = int(input("Enter Vibration: "))

if Temperature < 80 and Vibration < 5:
  print("Normal")

elif (Temperature >= 80 and Temperature <= 100) or (Vibration >= 5 and Vibration <= 8):
  print("Warning")

elif Temperature > 100 or Vibration > 8:
  print("Critical")

else:
  print("Inspection required")

4. **<u>CODE :</u>**

d. **EQUIPMENT DATABASE**

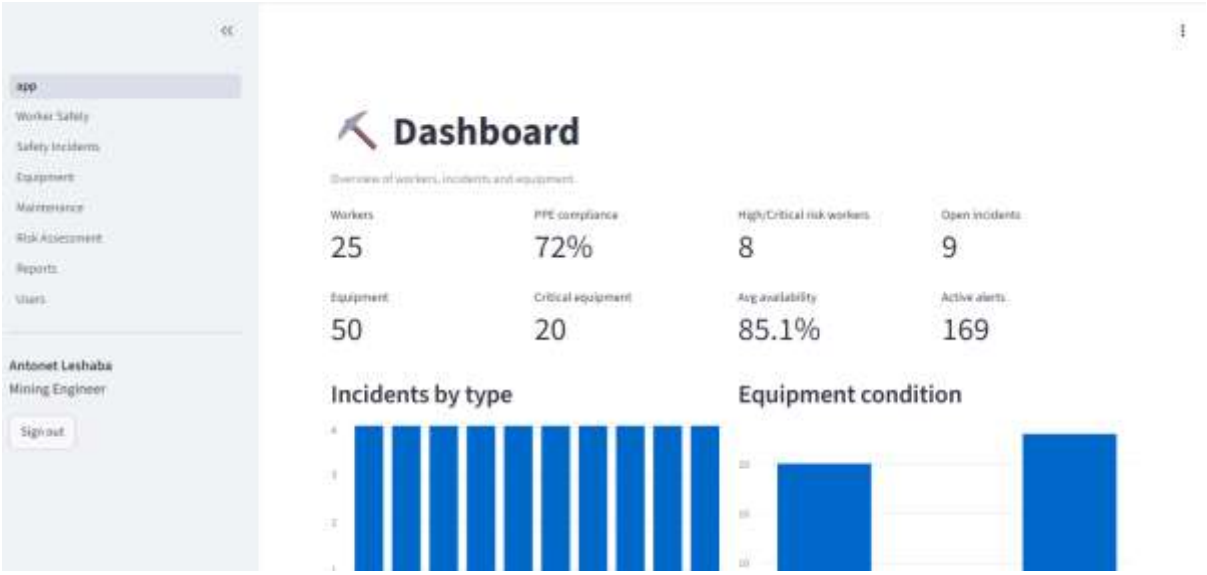
```
Equipment_ID= input("Enter Equipment ID:")
Equipment_type = input("Enter Equipment Type:")
Manufacture = input("Enter Manufactor:")
Operating_hrs= input("Enter Operating hours:")
Temparature = input("Enter Temperature:")
Vibration= input("Enter Vibration:")
Fuel_Consumption= input("Enter fuel consumption:")
Brake_Status=str(input("Enter Brake status"))
Tyre_Status=str(input("Enter Tyre status"))
Maintenance_Status=str(input("Enter Maintenance status"))
Engine_Status=str(input("Enter Engine status"))
Downtown=int(input("Enter Down time"))
```

e. **Equipment Condition Monitoring**

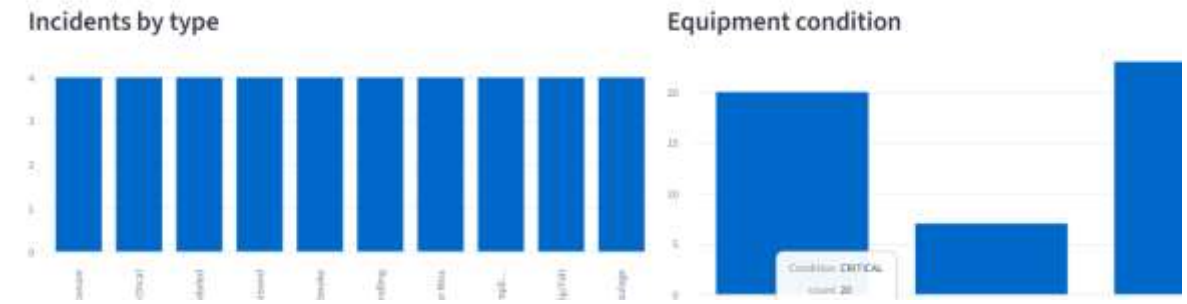
```
# EQUIPMENT CONDITION MONITORING
Temperature = int(input("Enter Temparature"))
Vibration= int(input("Enter Vibration"))
if Temperature <80 and Vibration < 5:
   print ("Normal")
elif Temperature >= 80 and Temperature <= 100 or Vibration >= 5 and Vibration <=
8 :
   print("Warning")
elif Temperature > 100 or Vibration > 8:
   print ("Critical")
else:
   print("Inspection required")
```

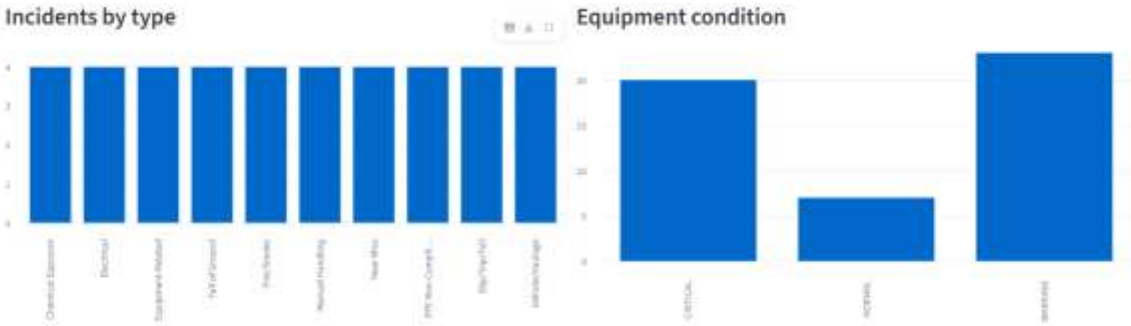
5. GRAPHICAL USER INTERPHASE FOR EQUIPMENT

MAIN DASHBOARD FOR EQUIPMENT

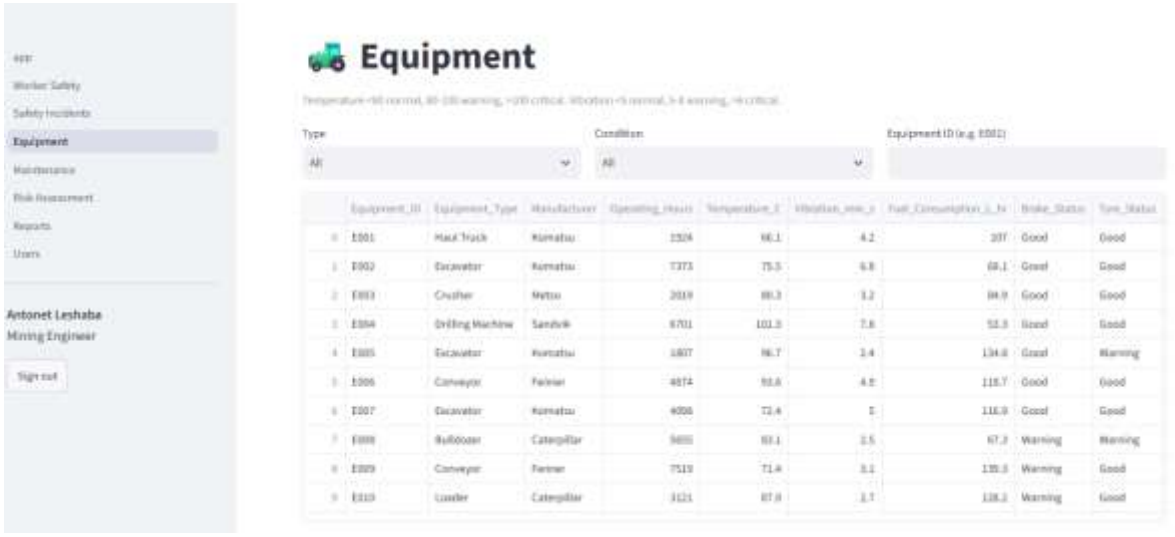


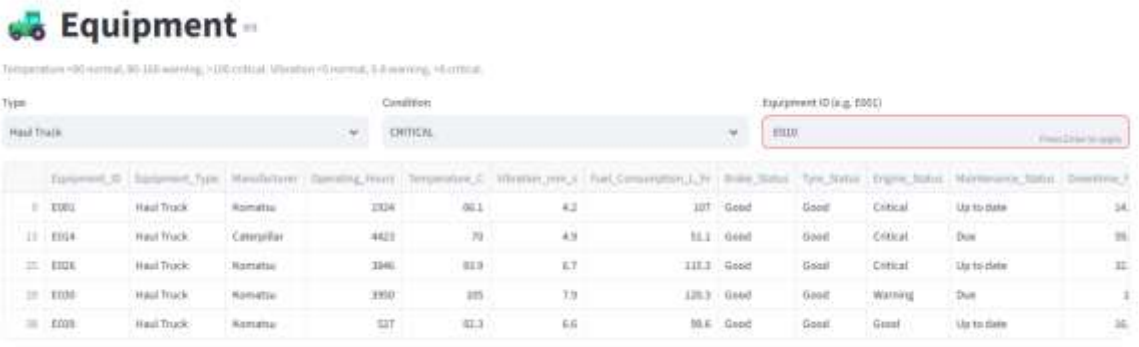






EQUIPMENT DATABASE



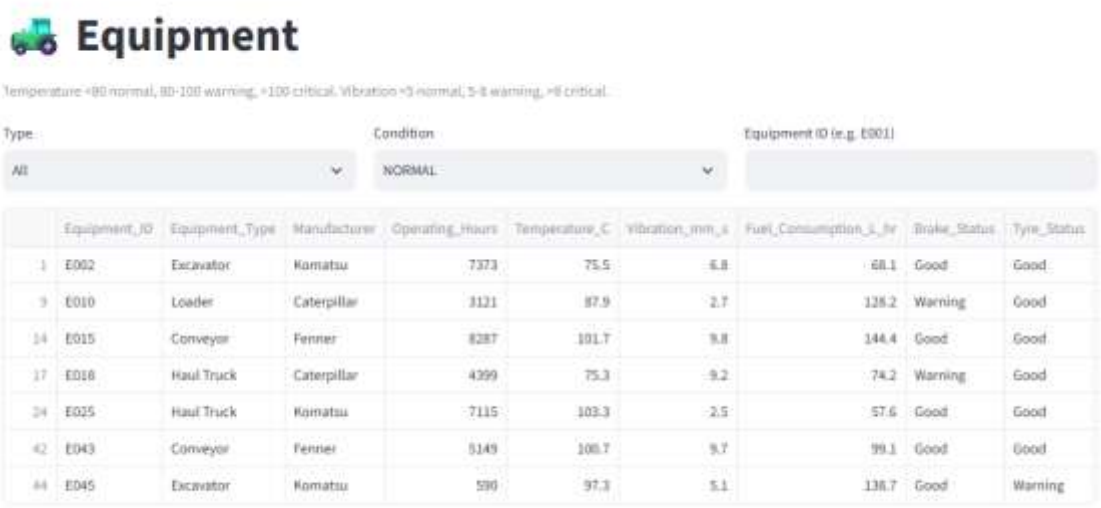


STATUS OF THE EQUIPMENT

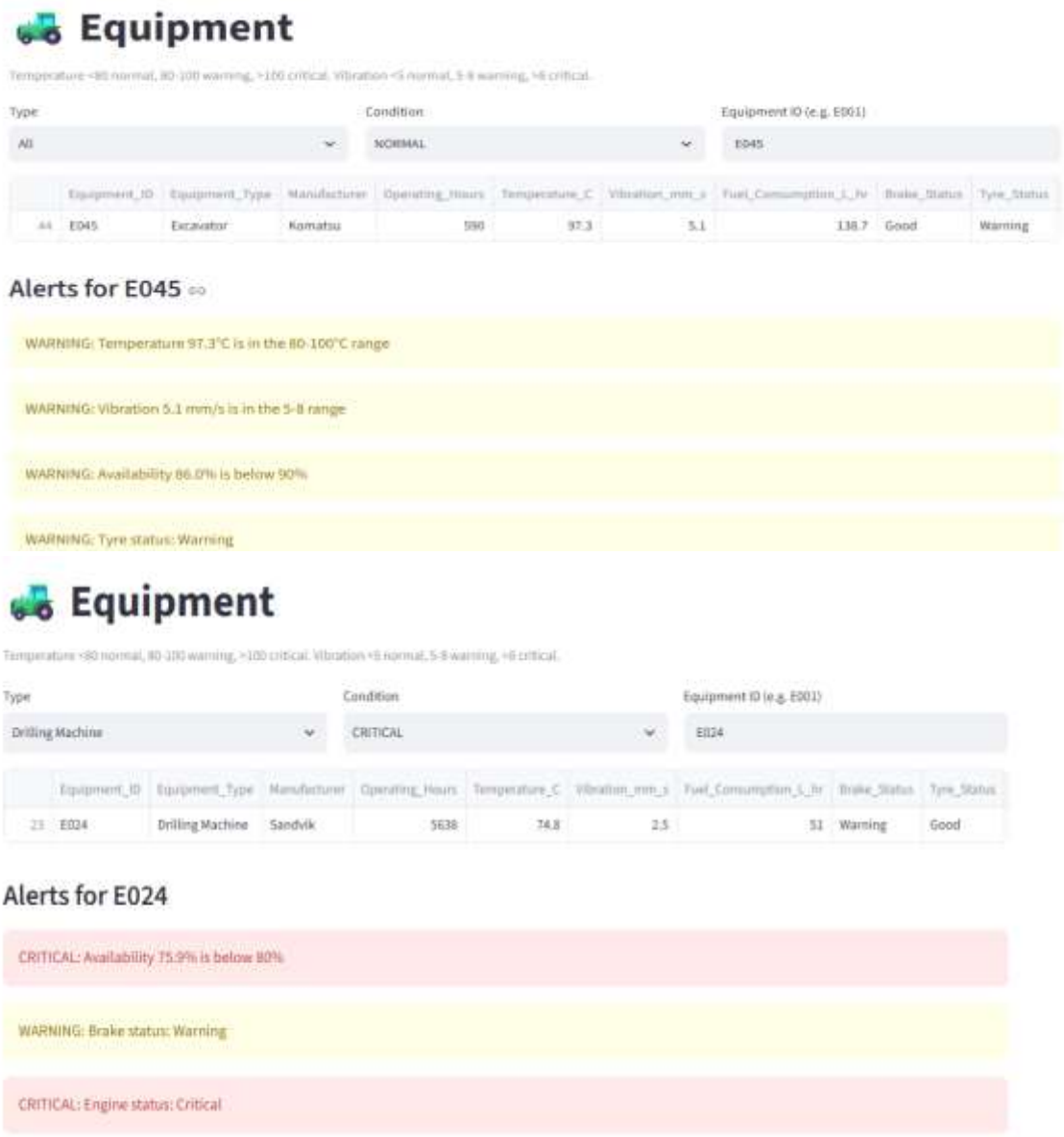
Equipment ID (e.g. E001)

Maintenance_Status	Downtime_hr	Availability_percent	Condition
Up to date	14.9	93.5	CRITICAL
Due	59.1	75	CRITICAL
Up to date	32.3	85.6	CRITICAL
Due	11	93.6	CRITICAL
Up to date	16.8	92.4	CRITICAL

CHECKING EQUIPMENT CONDITION



CHECKING ALERTS



c. Safety Incidents Database

Pseudocode: Main Menu

```
BEGIN Safety Incidents Database
    LOOP
        DISPLAY "--- SAFETY INCIDENTS DATABASE ---"
        DISPLAY "1. Add Incident"
        DISPLAY "2. View Incidents"
        DISPLAY "3. Search Incidents"
        DISPLAY "4. Filter Incidents"
        DISPLAY "5. Categorize Incidents"
        DISPLAY "6. Count Incidents"
        DISPLAY "7. Analyse Incident Trends"
        DISPLAY "8. Exit"
```

```
        PROMPT "Select an option:"
        INPUT   choice

        IF choice = 1 THEN
            CALL Add_Incident
        ELSE IF choice = 2 THEN
            CALL View_Incidents
        ELSE IF choice = 3 THEN
            CALL Search_Incidents
        ELSE IF choice = 4 THEN
            CALL Filter_Incidents
        ELSE IF choice = 5 THEN
            CALL Categorize_Incidents
        ELSE IF choice = 6 THEN
            CALL Count_Incidents
        ELSE IF choice = 7 THEN
            CALL Analyse_Trends
        ELSE IF choice = 8 THEN
            DISPLAY "Thank you for using the Safety Incidents Database."
            DISPLAY "Stay safe!"
            STOP
        ELSE
            DISPLAY "Invalid option. Please choose 1-8."
        END IF
    END LOOP
END
```

CODE

```
incidents = []


def add_incident():


  print("\n___ ADD SAFETY INCIDENT ___")


  Incident_ID = input("Insert Incident ID: ")

  Date = input("Insert Date (DD/MM/YY): ")

  Time = input("Insert Time: ")

  Shift = input("Insert Shift (Night/Day): ")

  Location = input("Insert Location (Mine Site): ")
```

```
Department = input("Insert Department: ")
Incident_type = input(
    "Insert Incident Type (Fall of Ground, Machinery Accident, etc.): "
)
Severity = input(
    "Enter Severity Level (Critical, High, Medium, Low): "
)
Lost_time = input(
    "Enter Lost-Time Injury Status (Yes/No): "
)
Cause = input("What is the Cause: ")
Corrective_action = input(
    "Enter Corrective Action Priority (High, Medium, Low): "
)
Status = input("Enter Incident Status (Open/Closed): ")

incident = {
    "ID": Incident_ID,
    "Date": Date,
    "Time": Time,
    "Shift": Shift,
    "Location": Location,
    "Department": Department,
    "Type": Incident_type,
    "Severity": Severity,
```

```
        "Lost Time": Lost_time,
        "Cause": Cause,
        "Corrective Action": Corrective_action,
        "Status": Status
    }

    incidents.append(incident)

    print("\nIncident added successfully.")

def view_incidents():

    print("\n___ ALL SAFETY INCIDENTS ___")

    if len(incidents) == 0:
        print("No incidents have been recorded.")
    else:
        for incident in incidents:
            print(incident)

def search_incidents():

    print("\n___ SEARCH INCIDENTS ___")

    search = input("Enter Incident ID: ")
```

```
    found = False

    for incident in incidents:
        if search == incident["ID"]:
            print("Incident found")
            print(incident)
            found = True

    if found == False:
        print("No matching incidents found.")

def filter_incidents():

    print("\n___ FILTER INCIDENTS ___")
    print("1. Filter by Severity")
    print("2. Filter by Shift")
    print("3. Filter by Incident Type")
    print("4. Filter by Department")
    print("5. Filter by Location")

    choice = input("Choose a filter: ")

    if choice == "1":
        value = input("Enter Severity: ")
```

```
    for incident in incidents:
        if incident["Severity"].lower() == value.lower():
            print(incident)

elif choice == "2":
    value = input("Enter Shift: ")

    for incident in incidents:
        if incident["Shift"].lower() == value.lower():
            print(incident)

elif choice == "3":
    value = input("Enter Incident Type: ")

    for incident in incidents:
        if incident["Type"].lower() == value.lower():
            print(incident)

elif choice == "4":
    value = input("Enter Department: ")

    for incident in incidents:
        if incident["Department"].lower() == value.lower():
            print(incident)
```

```
    elif choice == "5":
        value = input("Enter Location: ")

        for incident in incidents:
            if incident["Location"].lower() == value.lower():
                print(incident)

    else:
        print("Invalid choice.")

def categorize_incidents():

    print("\n___ INCIDENT CATEGORIES ___")

    if len(incidents) == 0:
        print("No incidents recorded.")
        return

    fall = 0
    machinery = 0
    fire = 0

    for incident in incidents:
```

```
        if incident["Type"].lower() == "fall of ground":
            fall = fall + 1

        elif incident["Type"].lower() == "machinery accident":
            machinery = machinery + 1

        elif incident["Type"].lower() == "fire":
            fire = fire + 1

    print("Fall of Ground:", fall)
    print("Machinery Accident:", machinery)
    print("Fire:", fire)

def count_incidents():

    print("\n___ INCIDENT COUNT ___")

    total = len(incidents)

    print("Total number of incidents:", total)

    critical = 0
    high = 0
    medium = 0
    low = 0
```

```
    for incident in incidents:

        severity = incident["Severity"].lower()

        if severity == "critical":
            critical += 1

        elif severity == "high":
            high += 1

        elif severity == "medium":
            medium += 1

        elif severity == "low":
            low += 1

    print("\nIncidents by Severity:")
    print("Critical:", critical)
    print("High:", high)
    print("Medium:", medium)
    print("Low:", low)

def analyse_trends():
```

```
print("\n___ INCIDENT TREND ANALYSIS ___")

print("Total number of incidents:", len(incidents))

fall = 0
machinery = 0
fire = 0

for incident in incidents:

    if incident["Type"].lower() == "fall of ground":
        fall = fall + 1

    elif incident["Type"].lower() == "machinery accident":
        machinery = machinery + 1

    elif incident["Type"].lower() == "fire":
        fire = fire + 1

print("\nIncident Trends:")

print("Falls of Ground:", fall)
print("Machinery Accidents:", machinery)
print("Fires:", fire)
```

```
    if fall > machinery and fall > fire:
        print("\nTrend: Falls of Ground are the most common.")

    elif machinery > fall and machinery > fire:
        print("\nTrend: Machinery Accidents are the most common.")

    elif fire > fall and fire > machinery:
        print("\nTrend: Fires are the most common.")

    else:
        print("\nTrend: There is no clear trend.")

print("\nSAFETY INCIDENTS DATABASE")
print("1. Add Incident")
print("2. View Incidents")
print("3. Search Incidents")
print("4. Filter Incidents")
print("5. Categorize Incidents")
print("6. Count Incidents")
print("7. Analyse Incident Trends")
print("8. Exit")

choice = input("\nSelect an option: ")

if choice == "1":
```

```
    add_incident()

elif choice == "2":
    view_incidents()

elif choice == "3":
    search_incidents()

elif choice == "4":
    filter_incidents()

elif choice == "5":
    categorize_incidents()

elif choice == "6":
    count_incidents()

elif choice == "7":
    analyse_trends()

elif choice == "8":
    print("\nThank you for using the Safety Incidents Database.")
    print("Stay safe!")

else:
```

```
print("\nInvalid option. Please choose 1-8.")
```

f. Code – EQUIPMENT AVAILABILITY

```
import pandas as pd

equipment_data = pd.read_csv("data/Equipment_data.csv")

print("\n--- Equipment Maintenance Monitoring ---")
Equipment_ID = input("Enter Equipment ID (e.g. E001): ").strip()
equipment = equipment_data[
  equipment_data["Equipment_ID"].astype(str).str.strip() ==
Equipment_ID
]
if equipment.empty:
  print("\nERROR: Equipment ID not found.")
else:
  machine = equipment.iloc[0]

  Equipment_Type = machine["Equipment_Type"]
  Manufacturer = machine["Manufacturer"]
  Operating_hrs = machine["Operating_Hours"]
  Downtime_hrs = machine["Downtime_hr"]
  Maintenance_Status = machine["Maintenance_Status"]

  Alerts = int(input("Enter Number of Alerts: "))

  Availability = (
    Operating_hrs /
    (Operating_hrs + Downtime_hrs)
  ) * 100

  print("\n--- Equipment Information ---")

  print("Equipment ID:", Equipment_ID)
  print("Equipment Type:", Equipment_Type)
```

```
    print("Manufacturer:", Manufacturer)
    print("Operating Hours:", Operating_hrs)
    print("Downtime:", Downtime_hrs)
    print("Maintenance Status:", Maintenance_Status)
    print("Number of Alerts:", Alerts)
    print("Equipment Availability:", round(Availability, 2), "%")

if Maintenance_Status == "Overdue":
    print("WARNING: Equipment is overdue for maintenance.")
elif Maintenance_Status == "Due":
        print("WARNING: Equipment is due for maintenance.")
elif Maintenance_Status == "Good":
    print("Equipment maintenance is up to date.")
else:
    print("Invalid maintenance status.")

if Downtime_hrs == 100:
    print("WARNING: Equipment has excessive downtime.")
elif Downtime_hrs >= 100:
    print("Critical: Equipment has excessive downtime.")
else:
    print("Equipment downtime is acceptable.")

if Alerts >= 3:
    print("WARNING: Equipment has repeated alerts.")
else:
    print("Equipment alerts are normal.")
# Check availability
if Availability == 90:
    print("WARNING: Equipment has low availability.")
elif Availability >= 100:
    print("Critical: Equipment has low availability.")
else:
    print("Equipment availability is good.")
```

1. Flowchart: Main Menu- Safety incidents

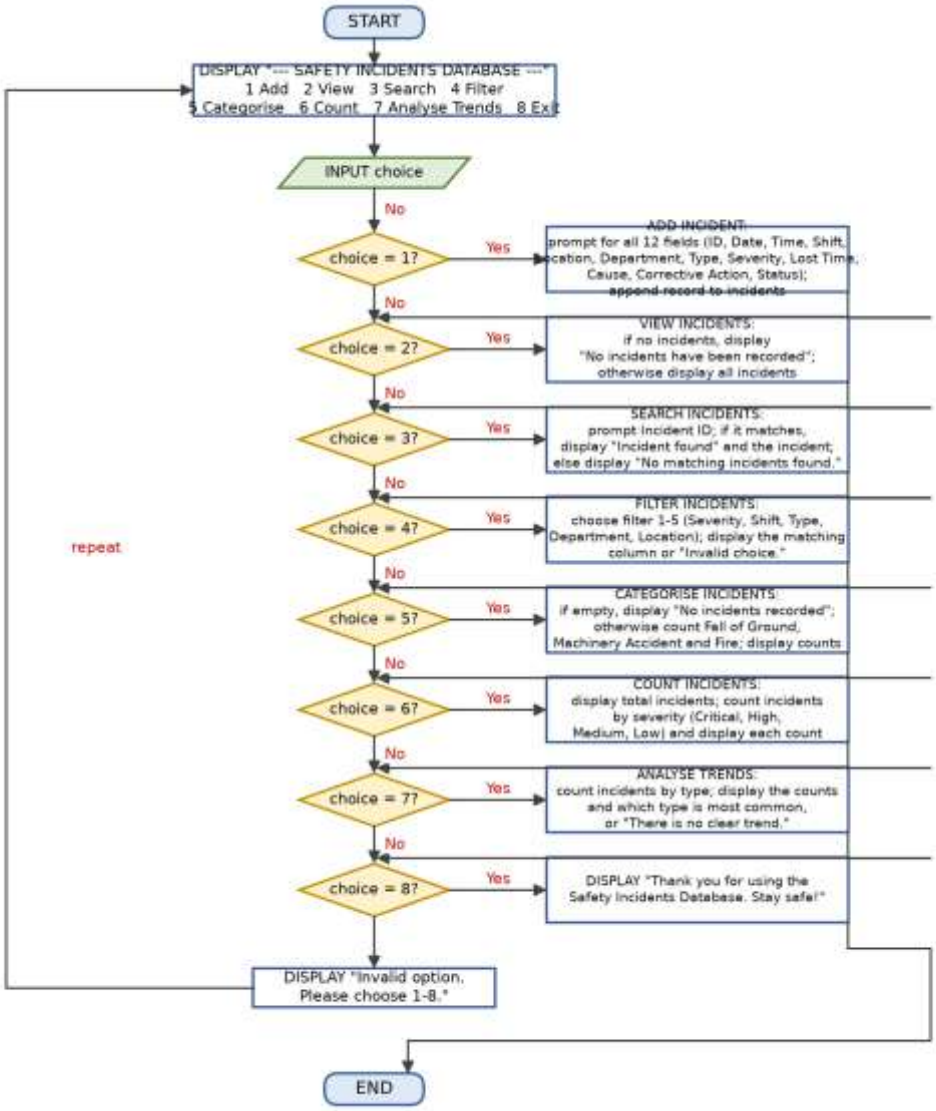


Figure 2. Flowchart of the Safety Incidents Database main menu

Pseudocode: Add Incident

```
MODULE Add_Incident
    DISPLAY "--- ADD SAFETY INCIDENT ---"
    PROMPT "Insert Incident ID:"
    INPUT   Incident_ID
    PROMPT "Insert Date (DD/MM/YY):"
    INPUT   Date
    PROMPT "Insert Time:"
    INPUT   Time
    PROMPT "Insert Shift (Night/Day):"
    INPUT   Shift
    PROMPT "Insert Location (Mine Site):"
    INPUT   Location
    PROMPT "Insert Department:"
    INPUT   Department
```

```
    PROMPT "Insert Incident Type (Fall of Ground, Machinery Accident, etc.):"
    INPUT   Incident_type
    PROMPT "Enter Severity Level (Critical, High, Medium, Low):"
    INPUT   Severity
    PROMPT "Enter Lost-Time Injury Status (Yes/No):"
    INPUT   Lost_time
    PROMPT "What is the Cause:"
    INPUT   Cause
    PROMPT "Enter Corrective Action Priority (High, Medium, Low):"
    INPUT   Corrective_action
    PROMPT "Enter Incident Status (Open/Closed):"
    INPUT   Status

    Incident <- record with columns
                ID = Incident_ID, Date, Time, Shift, Location, Department,
                Type = Incident_type, Severity, Lost Time = Lost_time,
                Cause, Corrective Action = Corrective_action, Status
    APPEND Incident TO incidents
END MODULE
```

Flowchart: Add Incident

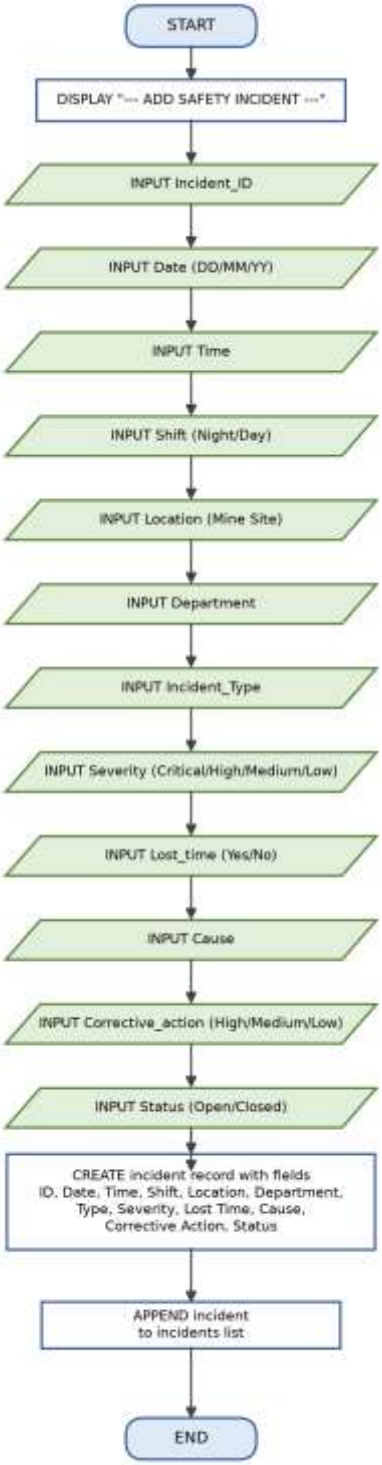


Figure 3. Flowchart of the Add Incident module

Pseudocode: Analyse Trends

```
MODULE Analyse_Trends
    DISPLAY "Total number of incidents:", LENGTH(incidents)
    fall      <- 0
    machinery <- 0
    fire      <- 0
    FOR EACH incident IN incidents DO
        IF incident.Type = "fall of ground" THEN
            fall <- fall + 1
        ELSE IF incident.Type = "machinery accident" THEN
            machinery <- machinery + 1
        ELSE IF incident.Type = "fire" THEN
            fire <- fire + 1
        END IF
    END FOR
    DISPLAY "Incident Trends:"
    DISPLAY "Falls of Ground:", fall
    DISPLAY "Machinery Accidents:", machinery
    DISPLAY "Fires:", fire

    IF fall > machinery AND fall > fire THEN
        DISPLAY "Trend: Falls of Ground are the most common."
    ELSE IF machinery > fall AND machinery > fire THEN
        DISPLAY "Trend: Machinery Accidents are the most common."
    ELSE IF fire > fall AND fire > machinery THEN
        DISPLAY "Trend: Fires are the most common."
    ELSE
        DISPLAY "Trend: There is no clear trend."
    END IF
END MODULE
```

Flowchart: Analyse Trends

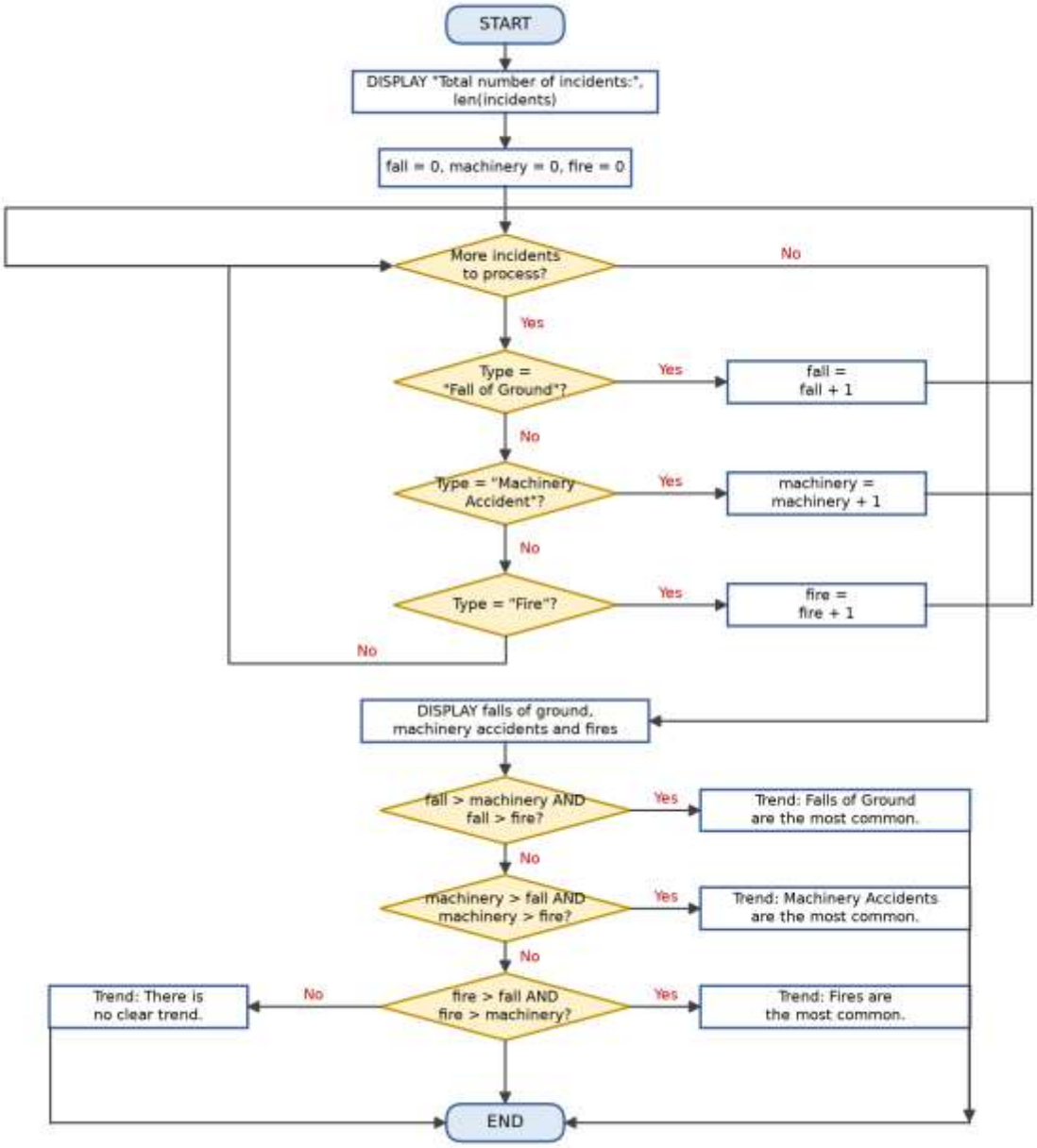


Figure 4. Flowchart of the Analyse Trends module

2. Flowchart- Equipment Availability

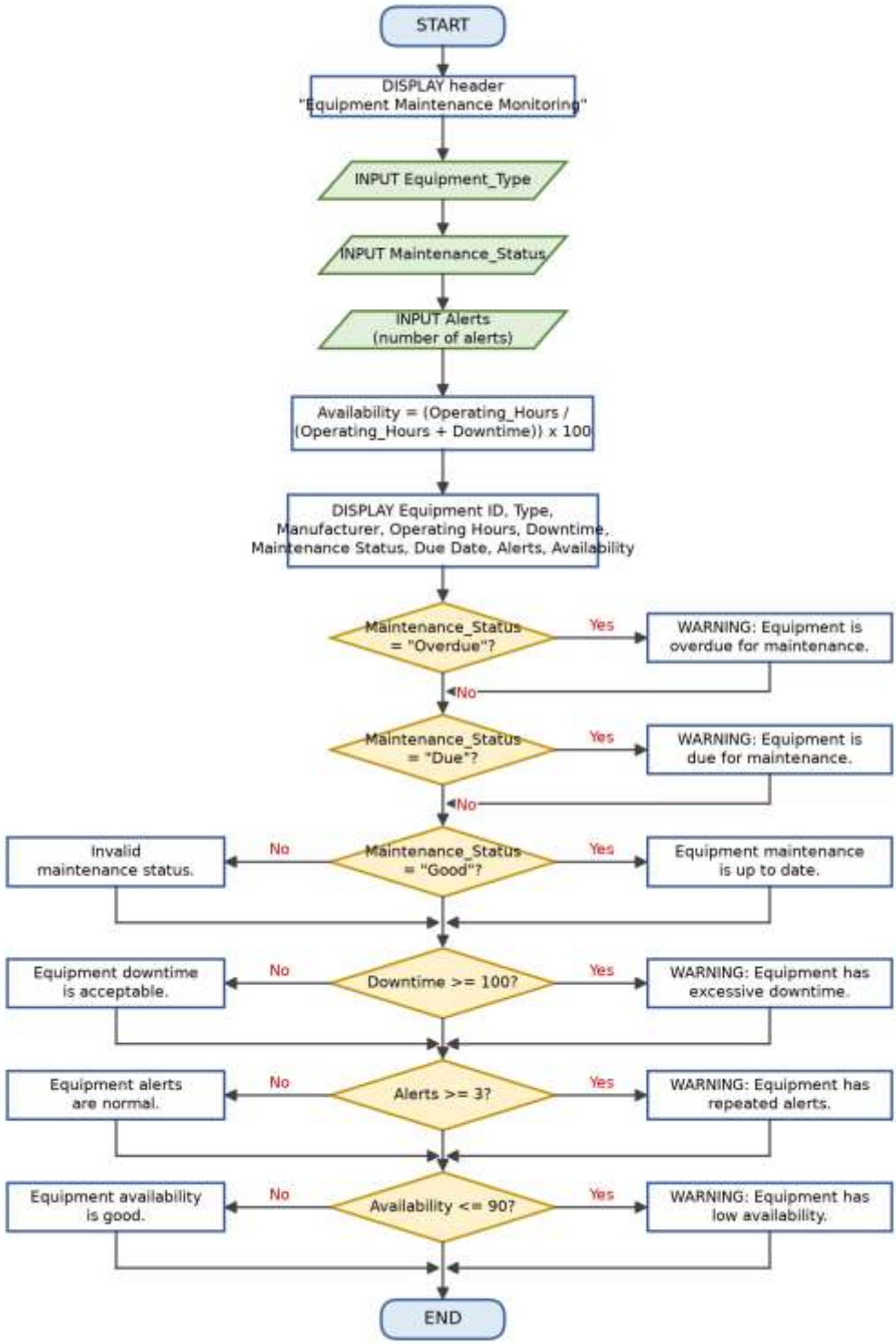


Figure 1. Flowchart of the Equipment Maintenance Monitoring program

Pseudocode

```
BEGIN Equipment Maintenance Monitoring
    DISPLAY "--- Equipment Maintenance Monitoring ---"
```

```
    PROMPT Equipment_Type        (Haul trucks, Loaders, Excavators, Drilling
                                 machines, Bulldozers, Scraper winches,
                                 Crushers, Conveyors)
    INPUT   Equipment_Type
    PROMPT Maintenance_Status    (Good, Due, Overdue)
    INPUT   Maintenance_Status
    PROMPT Alerts                (number of alerts)
    INPUT   Alerts

    Availability <- (Operating_Hours / (Operating_Hours + Downtime)) * 100

    DISPLAY "--- Equipment Information ---"
    DISPLAY "Equipment ID:", Equipment_ID
    DISPLAY "Equipment Type:", Equipment_Type
    DISPLAY "Manufacturer:", Manufacturer
    DISPLAY "Operating Hours:", Operating_Hours
    DISPLAY "Downtime:", Downtime
    DISPLAY "Maintenance Status:", Maintenance_Status
    DISPLAY "Maintenance Due Date:", Maintenance_Due
    DISPLAY "Number of Alerts:", Alerts
    DISPLAY "Equipment Availability:", Availability, "%"

    IF Maintenance_Status = "Overdue" THEN
        DISPLAY "WARNING: Equipment is overdue for maintenance."
    ELSE IF Maintenance_Status = "Due" THEN
        DISPLAY "WARNING: Equipment is due for maintenance."
    ELSE IF Maintenance_Status = "Good" THEN
        DISPLAY "Equipment maintenance is up to date."
    ELSE
        DISPLAY "Invalid maintenance status."
    END IF

    IF Downtime >= 100 THEN
        DISPLAY "WARNING: Equipment has excessive downtime."
    ELSE
        DISPLAY "Equipment downtime is acceptable."
    END IF

    IF Alerts >= 3 THEN
        DISPLAY "WARNING: Equipment has repeated alerts."
    ELSE
        DISPLAY "Equipment alerts are normal."
    END IF

    IF Availability <= 90 THEN
        DISPLAY "WARNING: Equipment has low availability."
    ELSE
        DISPLAY "Equipment availability is good."
    END IF
END
```

GRAPHICAL USER INTERFACE

PEOPLE LOGIN DETAILS



Users

Accounts and what each role can access.

	Name	Username	Role	Access
0	Samkelo Gumbi	samkelo	Administrator	dashboard, workers, incidents, equipment, maintenance, risk, reports, users
1	Nkwadi Lelaka	nkwadi	Safety Officer	dashboard, workers, incidents, risk, reports
2	Antonet Leshaba	antonet	Mining Engineer	dashboard, workers, equipment, maintenance, risk, reports
3	Thabo Dlamini	thabo	Maintenance Engineer	dashboard, equipment, maintenance, reports
4	Usenathi Kitshini	usenathi	Manager	dashboard, workers, incidents, equipment, maintenance, risk, reports



Safety Incidents 60

Search, filter and record incidents.

Search (ID, worker, cause...)

Severity: All | Shift: All | Incident type: All | Department: All | Status: All

60 of 60 incidents

	Incident_ID	Date	Time	Shift	Location	Department	Worker_ID	Worker_Name	Job_Role	Incident_Type	Severity	Injury_Status	Lost_Time	Cause
0	INC-001	2026/01/11	08:22	Day	Drilling Area	Mining	W007	Ayanda Dube	Dozer Operator	Vehicle/Haulage	Moderate	No	No	Operator error
1	INC-002	2026/01/16	10:19	Night	Loading Area	Mining	W014	Nokuthula Hadebe	Dozer Operator	Fall of Ground	Serious	No	No	Poor visibility
2	INC-003	2026/01/21	12:38	Day	Haul Road	Mining	W021	Nandi Mokoena	Dozer Operator	Chemical Exposure	Critical	No	No	Unsafe work pr
3	INC-004	2026/01/26	14:41	Night	Blasting Area	Mining	W028	Tshepo Cele	Dozer Operator	Equipment-Related	Minor	No	No	Loose material
4	INC-005	2026/01/31	16:59	Day	Open Pit	Mining	W035	Tumelo Ngcobo	Dozer Operator	Near Miss	Moderate	No	No	Poor housekee
5	INC-006	2026/02/05	18:57	Night	Conveyor Area	Processing	W042	Boitumelo Zulu	Conveyor Operator	Fire/Smoke	Serious	No	No	Operator error
6	INC-007	2026/02/10	06:04	Day	Screening Plant	Processing	W049	Mpho Dlamini	Crusher Operator	PPE Non-Compliance	Serious	No	No	Poor visibility
7	INC-008	2026/02/15	08:11	Night	Processing Plant	Processing	W056	Ayanda Nkosi	Plant Assistant	Manual Handling	Minor	No	No	Unsafe work pr

By severity

By cause

+ Add incident

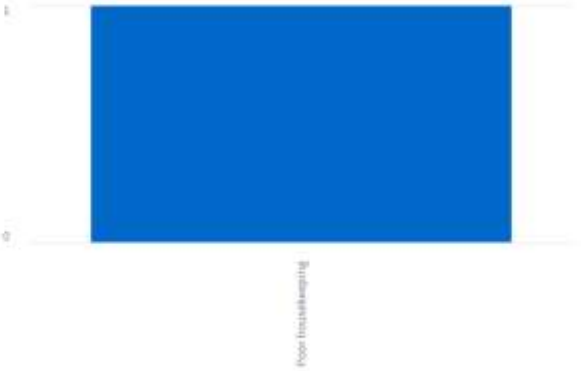
1 of 40 incidents

	Incident_ID	Date	Time	Shift	Location	Department	Worker_ID	Worker_Name	Job_Role	Incident_Type	Severi
34	INC-035	2026/06/30	06:20	Day	Screening Plant	Processing	W055	Kagiso Mthethwa	Plant Operator	Near Miss	Seriou

By severity



By cause



Severity: All

Shift: All

Incident type: PPE Non-Compliance

Department: All

Status: All

4 of 40 incidents

	Incident_ID	Date	Time	Shift	Location	Department	Worker_ID	Worker_Name	Job_Role	Incident_Type	Severi
6	INC-007	2026/02/10	06:04	Day	Screening Plant	Processing	W049	Masego Dlamini	Crusher Operator	PPE Non-Compliance	Seriou
16	INC-017	2026/04/01	12:14	Night	Loading Area	Mining	W024	Asanda Hadebe	Excavator Operator	PPE Non-Compliance	Moder
26	INC-027	2026/05/21	18:24	Night	Site-wide	Safety	W094	Thabiso Khumalo	Safety Officer	PPE Non-Compliance	Critic
36	INC-037	2026/07/10	10:34	Day	Conveyor Area	Maintenance	W069	Andile Dlamini	Maintenance Assistant	PPE Non-Compliance	Moder

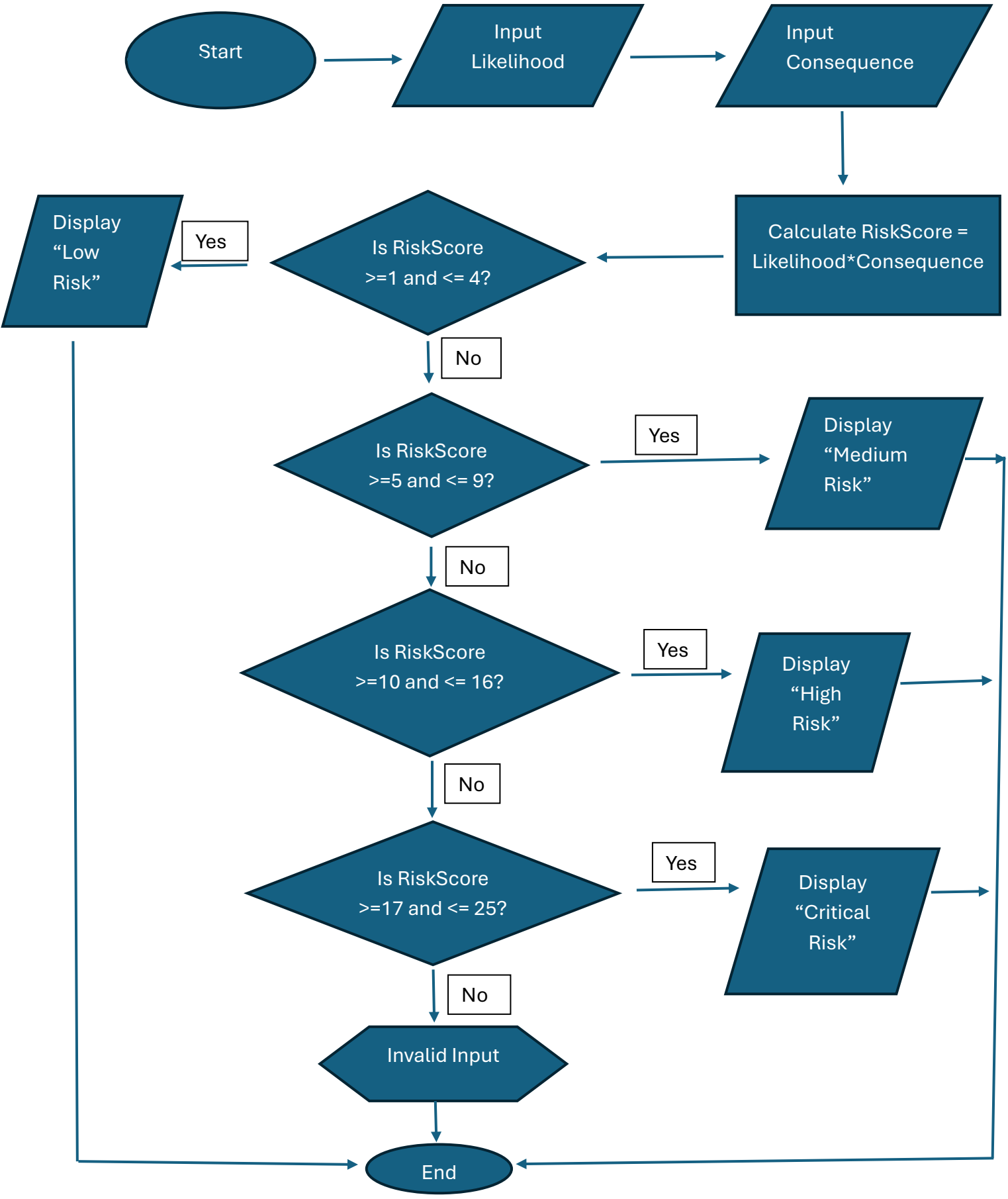
By severity



By cause



g. Risk Assessment Flowchart:



Pseudocode:

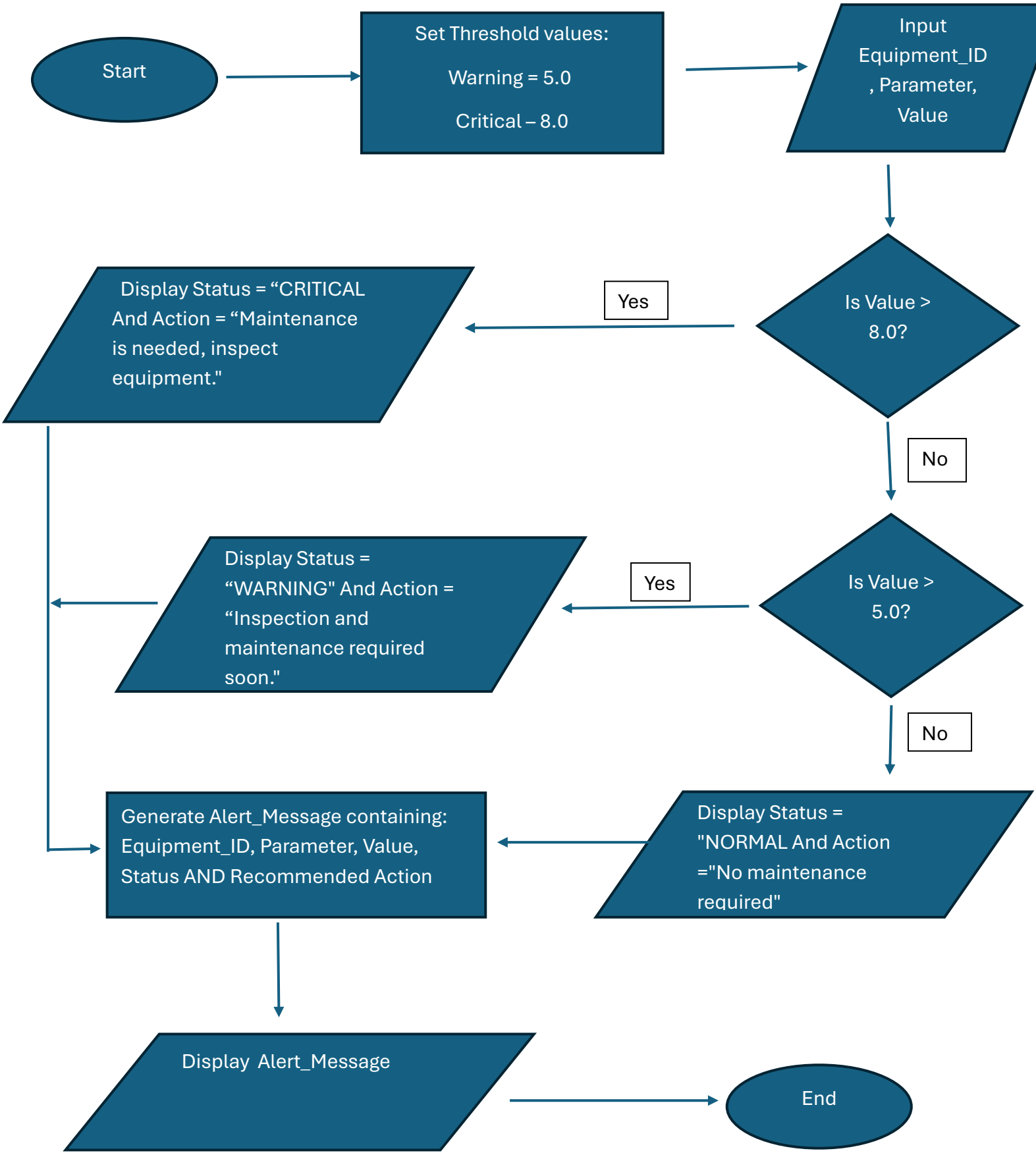
```
START
        INPUT Likelihood
        INPUT Consequence
       SET RiskScore = Likelihood * Consequence
       IF RiskScore >= 1 AND RiskScore <= 4 THEN
                DISPLAY "Low Risk"
      ELSE IF RiskScore >= 5 AND RiskScore <= 9 THEN
                DISPLAY "Medium Risk"
      ELSE IF RiskScore >= 10 AND RiskScore <= 16 THEN
                DISPLAY "High Risk"
      ELSE IF RiskScore >= 17 AND RiskScore <= 25 THEN
                DISPLAY "Critical Risk"
     ELSE
               DISPLAY "Invalid Input"
     END IF
STOP
```

Python Code:

```
Likelihood = int(input("Enter likelihood chances"))   #Ranges from 1 to 5

Consequence = int(input("Enter consequence chances"))  #Ranges from 1 to 5
Riskscore = Likelihood * Consequence                #Riskscore cannot exceed 25
if Riskscore >= 1 and Riskscore <= 4:
   print ("Low Risk")
elif Riskscore >= 5 and Riskscore <= 9:
   print("Medium Risk")
elif Riskscore >= 10 and Riskscore <= 16:
   print("High Risk")
elif Riskscore >= 17 and Riskscore <= 25:
   print("Critical Risk")
else:
   print("Invalid input")
```

h. Alerts and Notifications Flowchart:



PSEUDOCODE

```
START
        SET Warning Threshold = 5.0
        SET Critical Threshold = 8.0
        INPUT Equipment_ID
        INPUT Parameter
        INPUT Value
        IF Value > Critical Threshold THEN
            SET Status = “CRITICAL"
            SET Action = “Maintenance is needed, inspect equipment."
       ELSE IF Value > Warning Threshold THEN
            SET Status = “WARNING"
            SET Action = “Inspection and maintenance required soon."
        ELSE
            SET Status = "NORMAL"
            SET Action ="No maintenance required"
       END IF
       CREATE Alert_Message containing:
       Equipment_ID, Parameter, Value, Status AND Recommended Action
       DISPLAY Alert_Message
STOP
```

Python code:

```
thresholds_vibration = {"warning": 5.0, "critical": 8.0}
def generate_alert(equipment_id, parameter, value, thresholds):
    if value > thresholds["critical"]:
        status = "CRITICAL"
        action = "Maintenance is needed, inspect equipment."
    elif value > thresholds["warning"]:
        status = "WARNING"
        action = "Inspection and maintenance required soon."
    else:
        status = "NORMAL"
        action = "No maintenance required"

    alert_message = f"""


    ==============================================
    MINING MONITORING ALERT
    ==============================================
    [{status}]

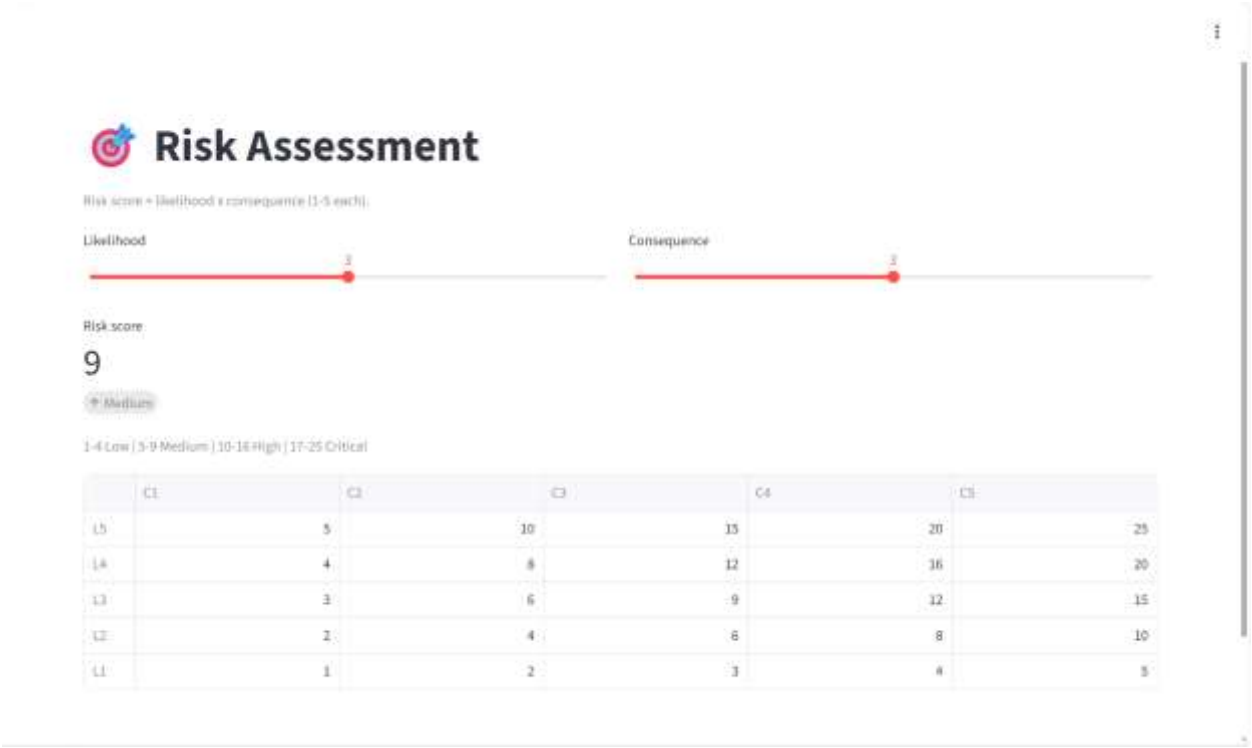
    Equipment ID: {equipment_id}
    Parameter: {parameter}
    Value: {value}
    Status: {status}

    Recommended Action:
    - {action}
    ==============================================
    """

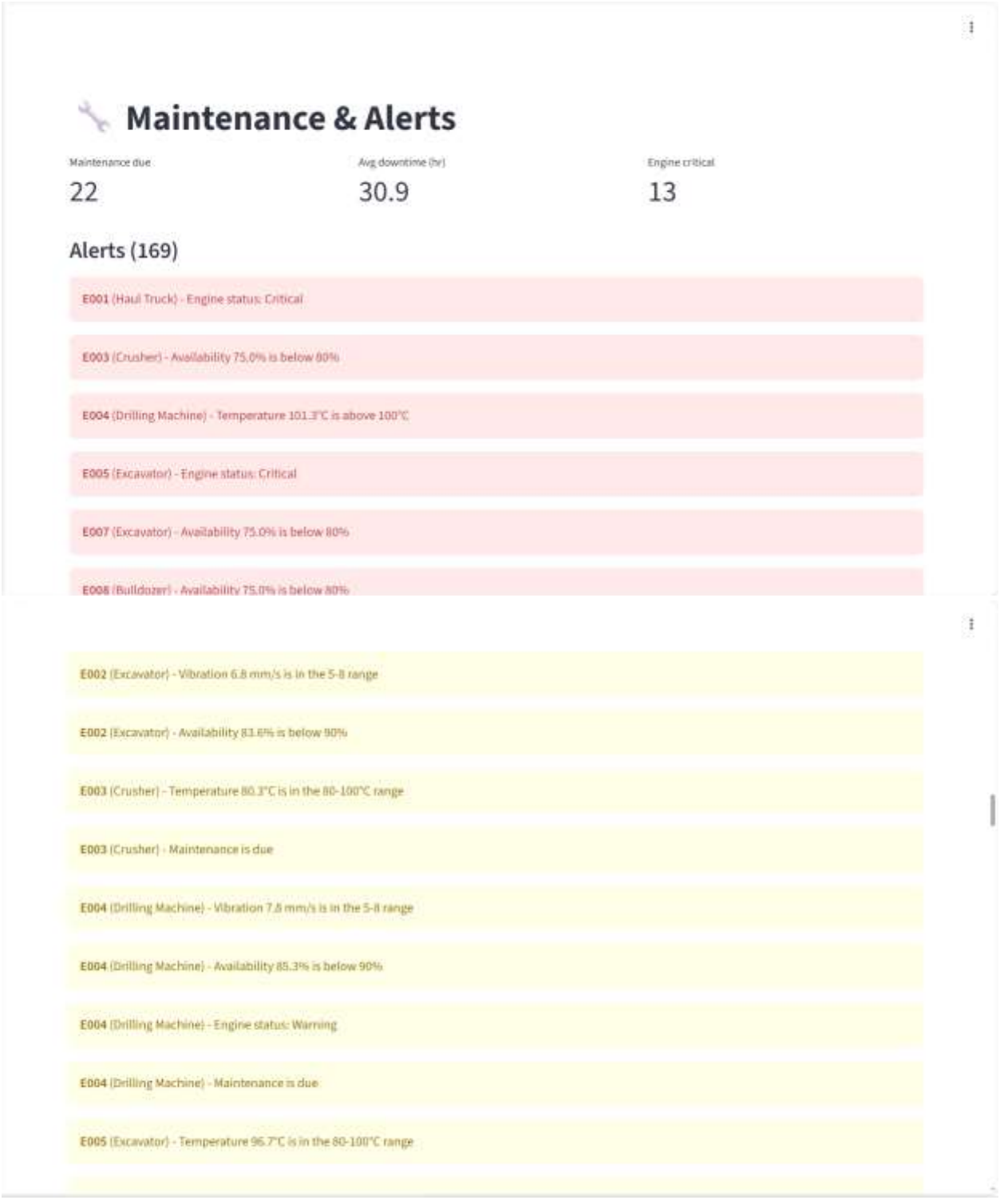
    return alert_message

print(generate_alert("TRK-012", "Vibration", 9.2, thresholds_vibration))
```

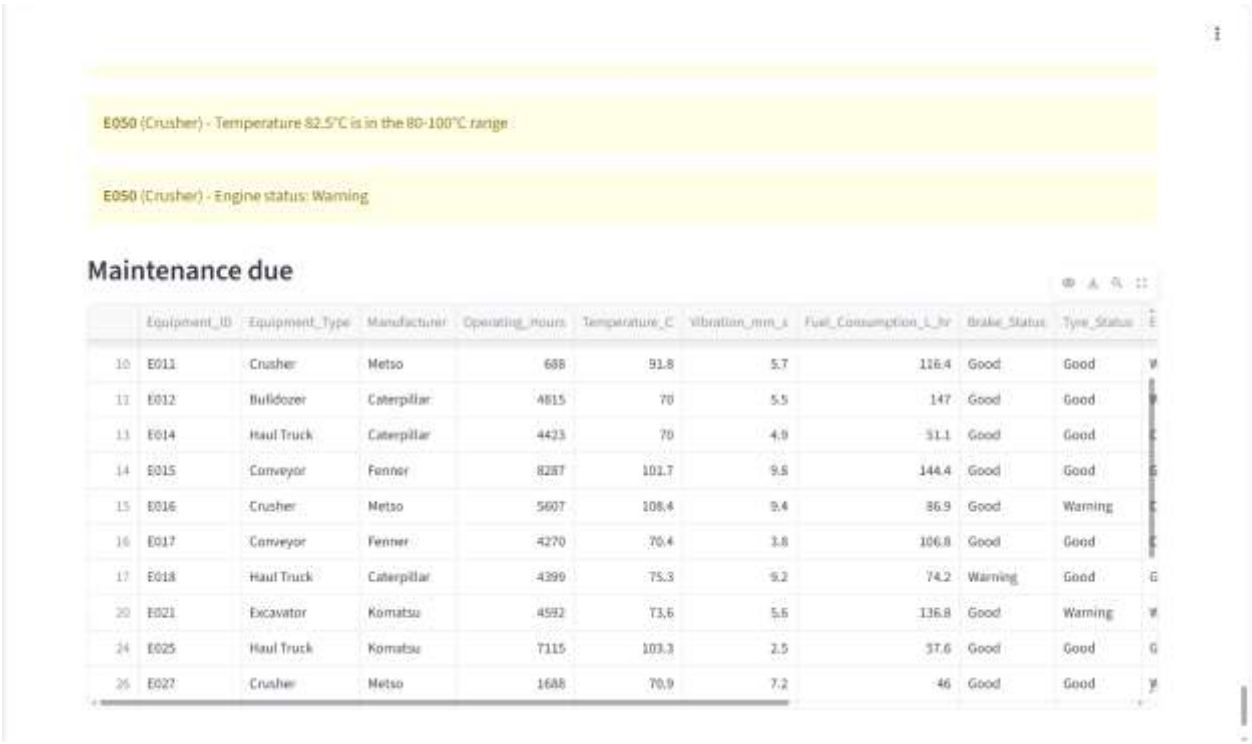
4. **GRAPHICAL USER INTERPHASE**



POSSIBLE ALERTS



MAINTANANCE DUE



	Equipment_ID	Equipment_Type	Manufacturer	Operating_Hours	Temperature_C	Vibration_mm_s	Fuel_Consumption_L_hr	Brake_Status	Tyre_Status
10	E011	Crusher	Metso	688	91.8	5.7	116.4	Good	Good
11	E012	Bulldozer	Caterpillar	4815	70	5.5	147	Good	Good
13	E014	Haul Truck	Caterpillar	4423	70	4.9	51.1	Good	Good
14	E015	Conveyor	Fenner	8287	101.7	9.6	144.4	Good	Good
15	E016	Crusher	Metso	5607	108.4	9.4	86.9	Good	Warning
16	E017	Conveyor	Fenner	4270	70.4	3.8	106.8	Good	Good
17	E018	Haul Truck	Caterpillar	4399	75.3	9.2	74.2	Warning	Good
20	E021	Excavator	Komatsu	4592	73.6	5.6	136.8	Good	Warning
24	E025	Haul Truck	Komatsu	7115	103.3	2.5	57.6	Good	Good
26	E027	Crusher	Metso	1688	70.9	7.2	46	Good	Good